

Software Assignment-2

Name	Entry No
Ahilaan Saxena	2023CS10076
Popat Nihal Alkesh	2023CS10058

Design Decisions:

1) Greedy Algorithm:

Classes involved

- Defined classes include `Gate`, `Pin`, `Network`, and `Queue`.
- A 'network' refers to a single wire connecting multiple pins.

Gate ordering for placement

- The aim is to minimize delay, which depends on the number of connections and gate delays.
- Gates are sorted based on the sum of gate delays of neighbouring (directly connected) gates.
- This sorting ensures wires with greater gate delays are minimized in length.

Greedy algorithm for gate placement

- After sorting, gates are passed to a greedy algorithm to find optimal placement.
- A set of perimeter points of the currently filled region is used to iterate over potential positions for gate placement.
- The cost function determines the best position based on minimizing the calculated cost.

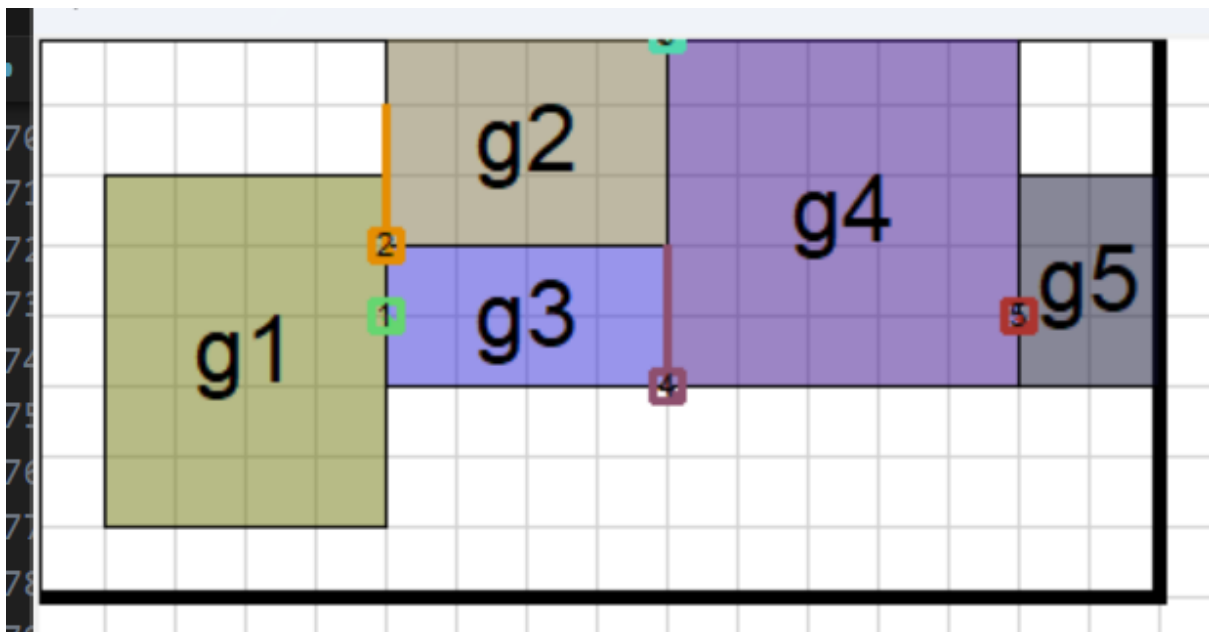
Cost function optimization

- Initially, the cost function was the sum of wire lengths of all networks connected to the gate being added.
- It was modified to the sum of `aggregate gate delay \* wire length` over all networks.
- Aggregate gate delay = sum of delays of all gates currently in the network.
- This modification prioritizes minimizing wire length for networks with higher gate delays to reduce critical path delay.

Critical path and delay calculation

- A recursive function was used initially to compute the maximum delay for each gate.
- This function checks the input gate and recursively calculates the maximum delay of connected gates through output pins.
- To optimize, the recursive function was converted to dynamic programming.
- Finally, the function was restructured iteratively to avoid issues like recursion depth limit. This was done by using stack. Recursion can be replicated in iteration by using stack.
- To check for infinite loops in the circuit, the constraint on the number of gates was used. We know that max number of gates = 1000. Thus, whenever the recursion depth (stack size in our case) will be >1000, we know it is going to be an infinite loop (since any gate can be visited at most once), thus raising "LoopException" in that case.

Figure 1: test case given on moodle: an optimal critical delay of 32 is obtained



Time complexity analysis:

No. of gates (N) ≤ 1000; Max width ≤ 100; No. of pins (P) ≤ 40000

1. Sorting the Gates:

- For each gate we precompute the sum of gate delays of directly connected gates when we read the input. Let g_i be the number of gates connected to gate G_i . We claim, $O(\sum g_i) = O(N)$
- This is true because if S_i is the subset of gates connected directly to G_i (gates in S_i are basically the children of G_i), there can be no other gate G_j such that any gate in S_i is directly connected to G_j . This is using the fact

that any input pin can receive input from at most one output pin. Thus, the union of S_i is the set of all gates and since the subsets are disjoint, $O(\sum g_i) = O(N)$

- Once we have these values precomputed for each gate, sorting them takes $O(N \cdot \log(N))$ time complexity. Thus, overall time complexity for this step: $O(N) + O(N \cdot \log(N)) = O(N \cdot \log(N))$

2. **Greedy Algorithm:**

- The overall process involves placing the gates one by one to the pre-existing gate structure by iterating over the gates directly connected to the current gate and following this recursively for every connected gate. This takes $O(N)$ amount of time as every gate is visited exactly once.
- Now, considering the operations within each cluster; let n and p denote the no. of gates and pins respectively in a cluster. Within each cluster, the following operations are performed:
 - Assigning a network to each pin: $O(p)$
 - Iterating through each gate in the cluster: $O(n)$
- Within each cluster, the following operations are performed:
 - Iterating through the entire current perimeter: $O(n \cdot 400)$
 - Inside each perimeter iteration, we iterate through the gate perimeter: $O(400)$
- Hence, the time complexity per cluster: $O(p + 160000 \cdot n^2)$
- Let C be the no. of clusters. This results in an overall time complexity of: $O(N \cdot \log(N) + N + C(p + n^2 \cdot 160000))$
- Since $C \cdot p = P$ and $C \cdot n = N$, we can simplify this expression to: $O(N \cdot \log(N) + N + P + 160000 \cdot N \cdot n)$, where n is the average no of gates per cluster (for single cluster, $n = N$) i.e. $O(P + 160000 \cdot N^2)$

3. **Finding Critical Path Delay:**

- If the DP state is already filled; we access the value in $O(1)$. If the DP state isn't filled, we call the function recursively for each gate. This means that the function will be called at most N number of times, where N is the total number of gates.
- Now for each gate, we iterate over all its pins and use the DP table to get the max value. This will take $O(P)$ time complexity, where P is the number of pins across all gates
- Overall time complexity becomes $O(P + N)$ which is approximately $O(P)$ as number of pins dominates number of gates in practical circuits.

Thus, the overall time complexity for all 3 steps: $O(P + 160000 \cdot N^2)$

Since P is bounded by 40000, for sufficient value of N , we can estimate this as: $O(160000 \cdot N^2)$

Worst Case Estimation: In the worst case, the total number of operations is thereby approximated to 16×10^{10} . In practice, the process took 5-10 minutes to complete for the worst-case scenario.

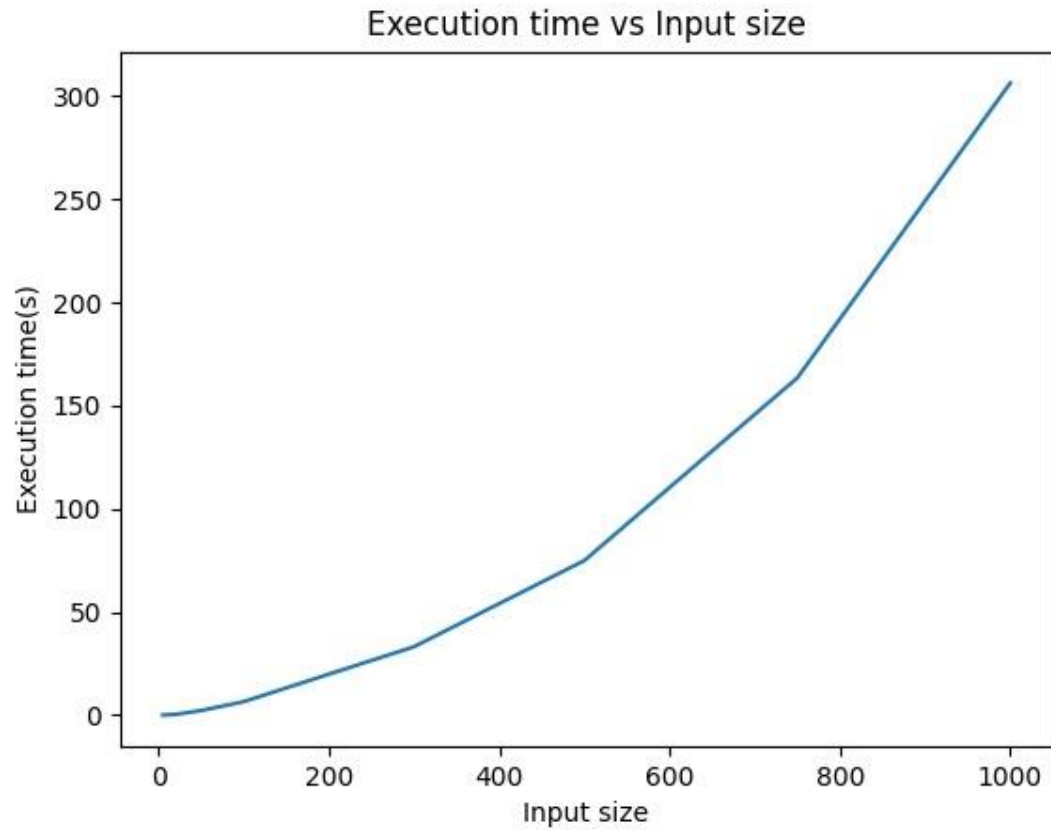


Figure 2: N vs t graph is approximately a parabola which verifies our hypothesis

No. of gates (N) in the cluster	Time taken (t)
5	0.08s
10	0.18s
20	0.40s
50	2.25s
100	6.58s
300	33.18s
500	74.96s
750	163.48s
1000	306.37s

Test Cases:

- For cases where number of pins might exceed 40000, we have modified the upper limit of the randint range as $\min(\text{height}, 20)$ to give the number of total pins less than this value
- For random cases: the gate delays and wire delays are chosen as random integers between 1 and 10, and 1 and 5 respectively. More over the generator works by choosing a random number of connections for each pin and generates that connection with a certain fixed probability depending on the test case.
- Cycles are avoided by choosing a gate having higher id as the sink for our current source gate in the iteration.
- Also, we ensure that each gate has at least one connection by choosing a single random connection during the first pass with 100% chance.

1) Linear 100

- Checks the packing for 100 gates, each of dimensions 10×10 , each having a single input and output pin located at their midpoints and the connections forming a straight line i.e. the output pin of the k th gate is connected to the input pin of $(k+1)$ th gate for all k .
- An idealization of symmetric circuits, it tests the ability of the algorithm to not overcomplicate simple circuits. For this purpose, we set the wire delay to a high value, compared to gate delay=1.
- As expected, the gates are connected linearly with total delay= $100 \times 1 = 100$ and zero wire delay.



2) Random 1000

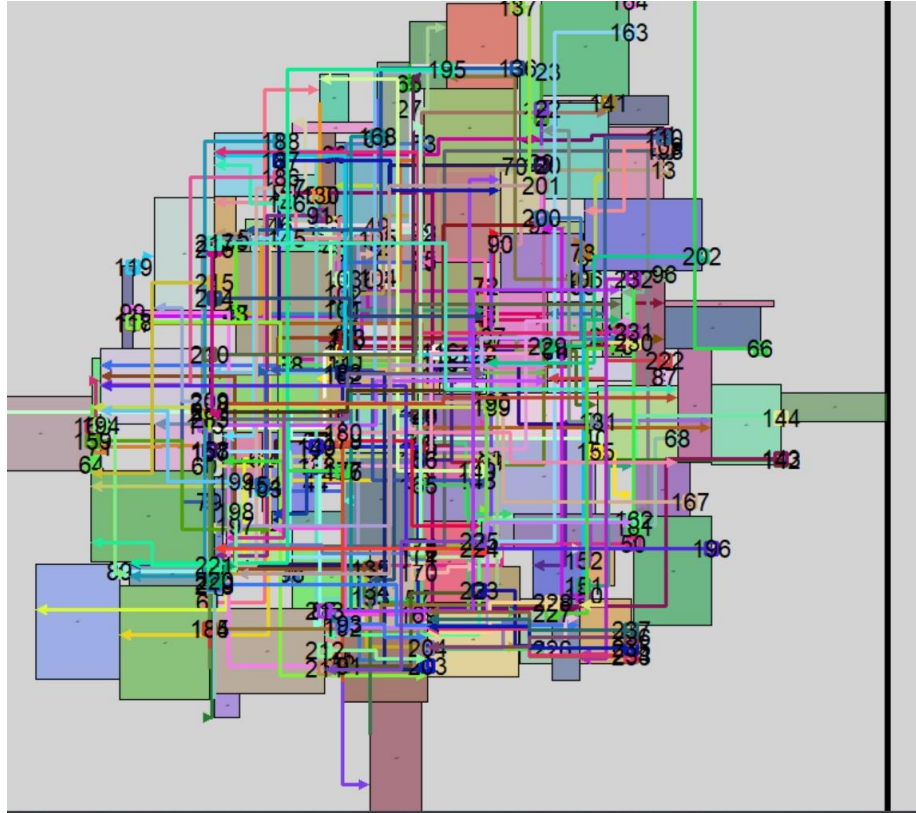
- Generates 1000 gates using randint function (dimension range 1 to 100) and having a random number of pins and connections among each other (upper limit of number of pins adjusted to $\min(\text{height}, 20)$)
- Chooses a random integer between 1 and 4 as the number of iterations for each pin and during each iteration, establishes a connection between this pin and some other gate of higher index with 40% chance, thus ensuring reasonable density with regards to practical circuits.

- Tests the ability of the algorithms to work on random circuits with no well-defined pattern of gate sizes and interconnections. It also gives a fair idea of the worst-case time complexity of the algorithm under the given constraints (took 3-5 minutes for different values)

(visualization not attached due to large number of gates)

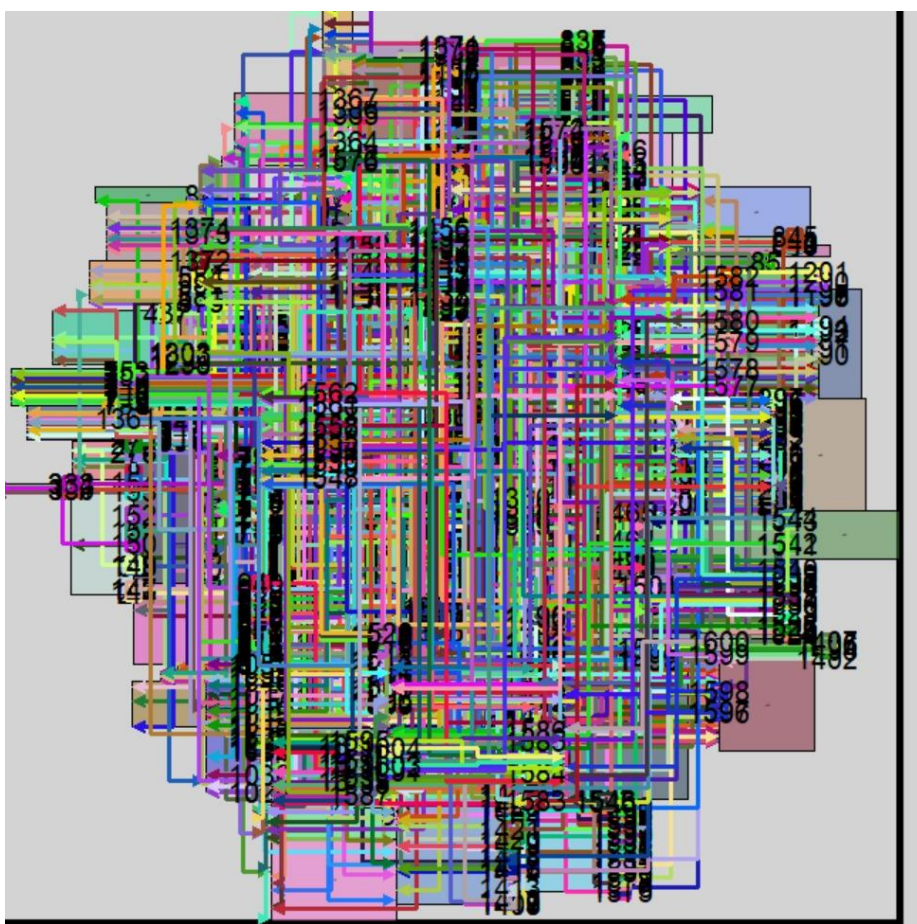
3) Sparsely connected gates

- Samples 100 gates of varying dimensions and randomly assigned pin numbers and defines connections such that each pin is connected to at most 2 other gates ($\text{no. of iterations} = \text{randint}(1, 2)$)
- For each iteration, the connection is established with a 5% chance, i.e. there is maximum ~10% chance ($0.95 * 0.95 = 0.9025$) that any pin will have at least 1 connection.
- Modelling circuits where the interconnections between gates are sparse.
- A point worth noting here is that the answer is significantly more when we sort by number of connections (~15000) compared to sorting by our custom method involving gate delays (~11000). This is because for lesser number of connections, the gate delays become significant and have greater effect on the output after sorting.



4) Densely Connected Gates

- Samples 100 gates of varying dimensions and randomly assigned pin numbers and defines connections such that each pin is connected to at most 4 other gates (no. of iterations=randint(1,4), same as random1000)
- For each iteration, the connection is established with a 60% chance, thus giving high density of connections.
- Modelling circuits where the interconnections between gates are dense.



5) Cycles

- Tests the ability of the algorithm to detect cycles in erroneous circuits and raise an exception accordingly.
- We created this by choosing the last gate and explicitly connecting it to some gate having a lower index, thus creating a cycle. As expected, the program detects path length >1000, thus raising an exception.

6) Powers of two

- Creates a gate structure in powers of two using 255 gates, i.e. the output of every gate is connected to inputs of 2 other gates, thus creating a perfect binary tree structure, analogous to circuits like the 64-bit adder (which has a perfect quaternary tree)
- We explicitly set the gate delay of the first gate in every level as 100 and remaining gate delays and wire delays as 1. This tests the ability of the algorithm to find the critical path correctly using the dynamic programming approach.
- As expected, we get a high critical delay of 816, though physically the gates don't form a binary tree arrangement. The visualization of the longest path verifies our critical path calculation because it goes through all the 8 gates having delay of 100 units each. The additional

delay of 16 units arises from the wire delay calculation for a network using bounding box method.

